

Algorithmen und Datenstrukturen



AVL-Bäume

AVL-Bäume

Das Problem mit binären Suchbäumen

Viele Operationen (Suchen, Einfügen, Löschen) sind abhängig von der Höhe h des Baums

Die Höhe ergibt sich durch die Einfügereihenfolge der Elemente

Im Worst Case (Einfügen einer sortierten Folge mit n Elementen) ergibt sich die Höhe n

Das Problem ist vergleichbar mit dem Worst Case beim Quick-Sort

AVL-Bäume

Lösungsansatz

- Ein AVL-Baum ist ein binärer Suchbaum, der bei jedem Einfügen oder Löschen „balanciert“ wird
- Benannt nach Georgi Maximowitsch **A**delson-**V**elski und Jewgeni Michailowitsch **L**andis
- Der Balancefaktor BF eines Knotens k ist definiert durch

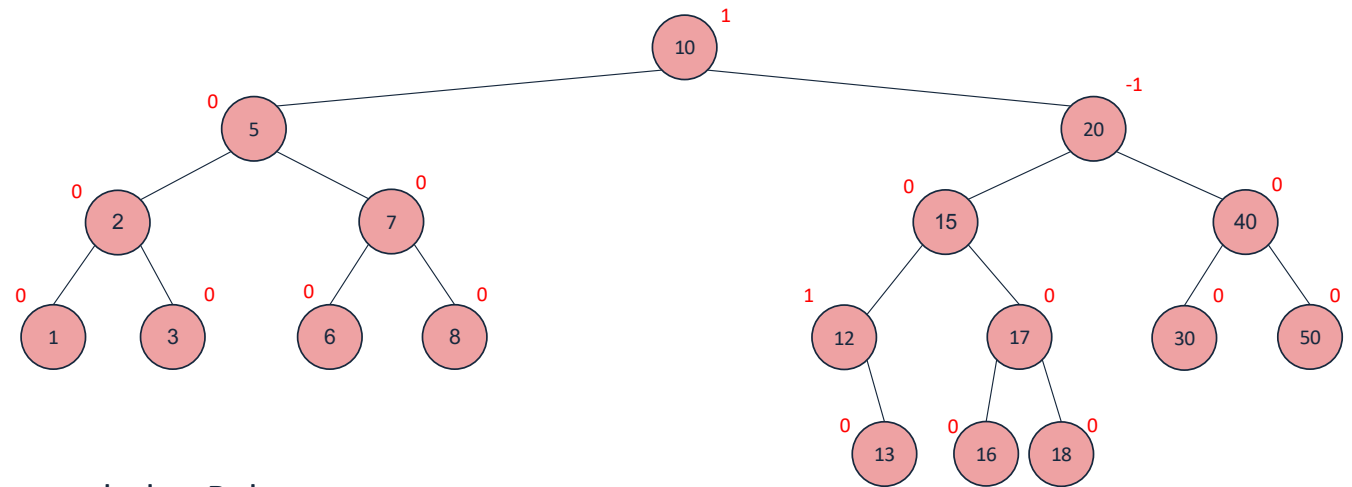
$$BF(k) = HEIGHT(k.right) - HEIGHT(k.left)$$

- Für jeden Knoten k des Baums muss gelten:

$$BF(k) \in \{-1, 0, 1\}$$

AVL-Bäume

Lösungsansatz



Die Datenstruktur für einen Knoten sollte auch den Balance-Faktor und einen Verweis zum Elternknoten enthalten:

- key*: Schlüsselwert des Knoten
- balance*: Schlüsselwert des Knoten
- left*: Verweis zum linken Nachfolger (oder NIL)
- right*: Verweis zum rechten Nachfolger (oder NIL)
- parent*: Verweis zum Elternknoten

AVL-Bäume

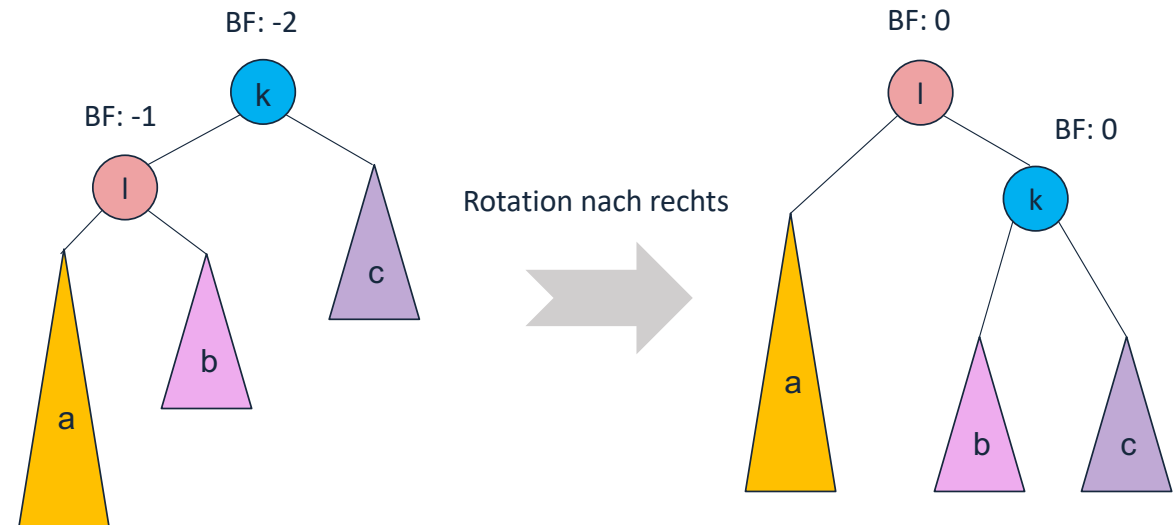
Balancieren durch Rotation

1. Beispiel

Knoten k verletzt das Balance-Kriterium und das Kind mit dem zu tiefen Teilbaum ist ausbalanciert oder in der gleichen Richtung unbalanciert, z.B.:

$$BF(k) < -1 \text{ und } BF(k.\text{left}) \leq 0$$

In diesem Fall kann die Balance durch eine einfache Rotation wieder hergestellt werden.



AVL-Bäume

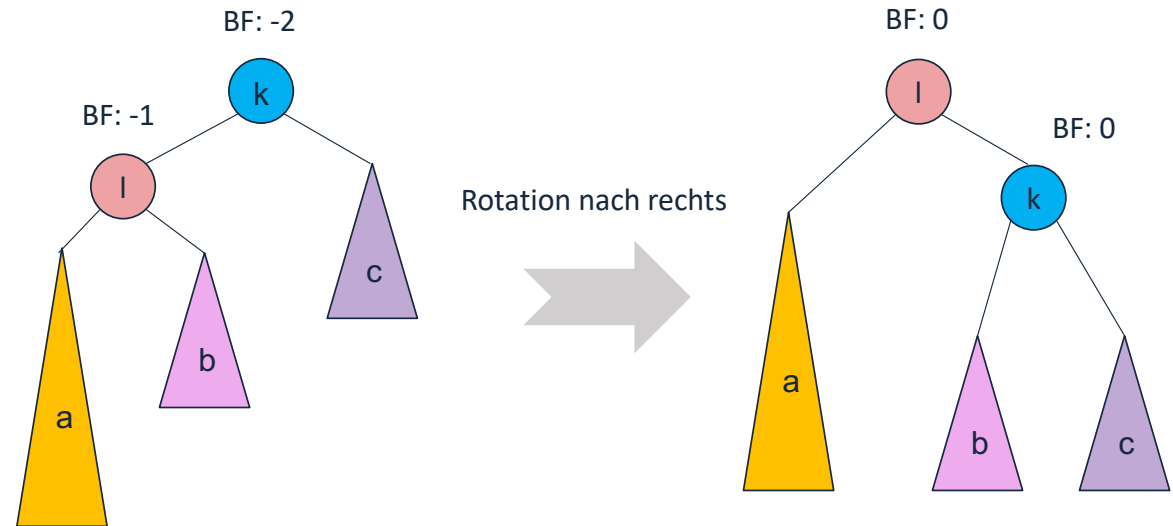
Balancieren durch Rotation

RIGHT-ROTATE(k)

```

1:  $l \leftarrow k.left$ 
2:  $l.parent \leftarrow k.parent$ 
3:  $k.left \leftarrow l.right$ 
4: if  $k.left \neq \text{NIL}$  then
5:    $k.left.parent = k$ 
6: end if
7:  $l.right \leftarrow k$ 
8:  $k.parent \leftarrow l$ 
9: if  $l.parent \neq \text{NIL}$  then
10:  if  $l.parent.right = k$  then
11:     $l.parent.right \leftarrow l$ 
12:  else if  $l.parent.left = k$  then
13:     $l.parent.left \leftarrow l$ 
14:  end if
15: end if
16:  $k.balance \leftarrow \text{HEIGHT}(k.right) - \text{HEIGHT}(k.left)$ 
17:  $l.balance \leftarrow \text{HEIGHT}(l.right) - \text{HEIGHT}(l.left)$ 
18: return  $l$ 

```



AVL-Bäume

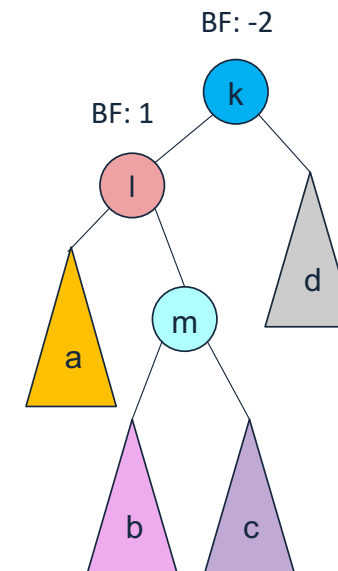
Balancieren durch Rotation

2. Beispiel

Knoten k verletzt das Balance-Kriterium und das Kind mit dem zu tiefen Teilbaum ist in der Gegenrichtung unbalanciert, z.B.:

$$BF(k) < -1 \text{ und } BF(k.\text{left}) > 0$$

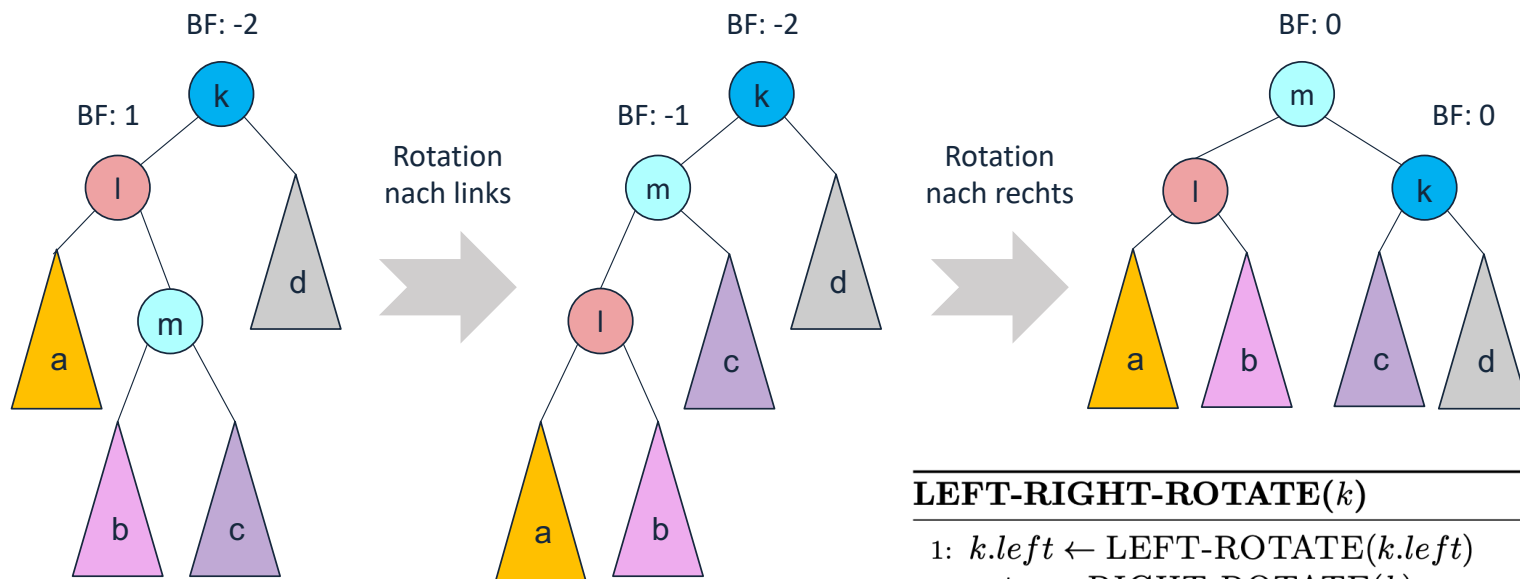
In diesem Fall stellt eine Doppelrotation die Balance wieder her.



AVL-Bäume

Balancieren durch Rotation

2. Beispiel



LEFT-ROTATE(k)

1: $k.left \leftarrow \text{LEFT-ROTATE}(k.left)$

2: **return** RIGHT-ROTATE(k)

AVL-Bäume

Balancieren nach Einfügen oder Löschen

Durch das Einfügen oder Löschen eines Knoten kann der Balance-Faktor mehrerer Knoten bis hinauf zur Wurzel gestört sein.

Die Korrektur erfolgt rekursiv aufsteigend zur Wurzel mit nebenstehendem Algorithmus.

AVL-BALANCE(T, k)

```
1:  $k.balance \leftarrow \text{HEIGHT}(k.right) - \text{HEIGHT}(k.left)$ 
2: if  $k.balance = -2$  then
3:   if  $\text{HEIGHT}(k.left.left) \geq \text{HEIGHT}(k.left.right)$  then
4:      $k \leftarrow \text{RIGHT-ROTATE}(k)$ 
5:   else
6:      $k \leftarrow \text{LEFT-RIGHT-ROTATE}(k)$ 
7:   end if
8: else if  $k.balance = 2$  then
9:   if  $\text{HEIGHT}(k.right.right) \geq \text{HEIGHT}(k.right.left)$  then
10:     $k \leftarrow \text{LEFT-ROTATE}(k)$ 
11:   else
12:     $k \leftarrow \text{RIGHT-LEFT-ROTATE}(k)$ 
13:   end if
14: end if
15: if  $k.parent \neq \text{NIL}$  then
16:    $\text{AVL-BALANCE}(k.parent)$ 
17: else
18:    $T.root \leftarrow k$ 
19: end if
```

AVL-Bäume

Einfügen in einen AVL-Baum

Zunächst wird der neue Knoten wie in einen binären Suchbaum als neues Blatt eingefügt.

Anschließend erfolgt das Balancieren vom Elternknoten des eingefügten Knotens bis zur Wurzel.

Das Balancieren geschieht mit $O(\log n)$.

Damit ist das Einfügen auch im Worst Case mit $O(\log n)$ durchführbar.

AVL-Bäume

Löschen in einen AVL-Baum

Zunächst wird der zu löschende Knoten wie in einen binären Suchbaum identifiziert und entfernt.

Anschließend erfolgt das Balancieren vom Elternknoten des gelöschten Knotens bis zur Wurzel.

Das Balancieren geschieht mit $O(\log n)$.

Damit ist das Löschen auch im Worst Case mit $O(\log n)$ durchführbar.

Vielen Dank für die Aufmerksamkeit!